

Rapport de Projet Académique

**CREATION D'UNE APPLICATION WEB-
VOYAGE CONNECT**

**Filière : Système d'Information et
Transformation Digitale**

Réalisé par :

GANDAHO MARIA

HOUNDJE AXEL

Encadrant :

**Mr. OUANAN
MOHAMMED**

Année Académique : 2025-2026

Table des Matières

Introduction.....	3
1. Contexte et Problématique.....	4
1.1 Contexte.....	4
1.2 Problématique.....	4
2. Objectifs du projet.....	5
2.1 Objectifs principaux.....	5
2.2 Objectifs spécifiques.....	5
3. Analyse et spécification des besoins.....	6
3.1 Besoins fonctionnels.....	6
3.2 Besoins non fonctionnels.....	6
4. Choix des technologies.....	8
4.1 Technologies Backend.....	8
4.2 Technologies Frontend.....	8
4.3 Base de données.....	9
4.4 Serveur d'application.....	9
4.5 Outils de développement.....	9
5. Architecture du système.....	11
5.1 Architecture MVC(Model-View-Controller).....	11
5.2 Flux de traitement d'une requête.....	11
6. Modélisation de la base de données.....	13
6.1 Diagramme de Cas d'Utilisation.....	13
6.2 Diagramme de Classe.....	13
7. Tests et validation.....	14
7.1 Tests unitaires.....	14
7.2 Tests d'intégration.....	14
8. Difficultés rencontrées et solutions.....	17
9. Conclusion.....	18
Annexes.....	19
Annexe A : Guide d'utilisation de l'application web.....	19

Introduction

Dans un contexte de digitalisation croissante du secteur touristique, les agences de voyage doivent s'adapter aux nouvelles attentes des consommateurs qui privilégient les réservations en ligne. Ce rapport présente le développement de **VoyageConnect**, une plateforme web complète permettant la gestion et la réservation de voyages en ligne.

VoyageConnect est une application web développée avec les technologies Java EE qui offre aux utilisateurs la possibilité de rechercher, comparer et réserver des voyages (vols, hôtels, circuits touristiques) de manière intuitive et sécurisée. Le projet inclut également un espace d'administration permettant à l'agence de gérer son catalogue d'offres et ses réservations sans oublier un guide d'utilisation détaillée exposant toutes les fonctionnalités et atouts du site.

1.Contexte et Problématique

1.1Contexte

Le marché du tourisme en ligne connaît une croissance exponentielle. Les voyageurs recherchent des plateformes leur permettant de planifier et réserver leurs voyages de manière autonome, à tout moment et depuis n'importe quel appareil. Les agences de voyage traditionnelles doivent donc proposer des solutions numériques pour rester compétitives.

1.2Problématique

Comment développer une plateforme web robuste et sécurisée permettant de :

- Gérer efficacement un catalogue de voyages diversifié (vols, hôtels, circuits) ;
- Offrir une expérience utilisateur fluide pour la recherche et la réservation ;
- Garantir la sécurité des transactions et des données personnelles ;
- Faciliter la gestion administrative des offres et des réservations

2.Objectifs du projet

1.1 Objectifs principaux

- Développer une application web complète pour une agence de voyage ;
- Permettre aux clients de rechercher et réserver des voyages en ligne ;
- Fournir aux administrateurs des outils de gestion efficaces ;
- Assurer la sécurité des transactions et des données utilisateurs ;
- Créer une interface responsive adaptée à tous les appareils

1.1 Objectifs spécifiques

- Implémenter un système d'authentification sécurisé avec gestion des rôles ;
- Développer un moteur de recherches multicritères performant ;
- Intégrer un système de paiement en ligne sécurisé ;
- Mettre en place un système de notifications automatiques ;
- Créer un tableau de bord pour les utilisateurs et les administrateurs

2. Analyse et spécification des besoins

2.1 Besoins fonctionnels

Pour les clients :

- Inscription et authentification sécurisée ;
- Recherche de voyages par critères multiples (destination, dates, budget, type) ;
- Consultation détaillée des offres (vols, hôtels, circuits) ;
- Réservation en ligne avec vérification des disponibilités
- Gestion du profil personnel ;
- Consultation de l'historique des réservations ;
- Modification ou annulation de réservations ;
- Paiement en ligne sécurisé ;
- Dépôt d'avis et recommandations ;
- Création d'itinéraires personnalisés

Pour les administrateurs :

- Gestion complète du catalogue d'offres (ajout, modification, suppression) ;
- Gestion des destinations et sites touristiques ;
- Suivi des réservations en temps réel ;
- Gestion des promotions et réductions ;
- Accès aux statistiques et rapports ;
- Gestion des avis clients

1.1 Besoins non fonctionnels

Performance :

- Temps de réponse inférieur à 2 secondes pour les recherches ;

- Support de multiples connexions simultanées ;
- Optimisation des requêtes à la base de données

Sécurité :

- Chiffrement des mots de passe ;
- Protection contre les injections SQL et XSS ;
- Sécurisation des transactions financières ;
- Gestion sécurisée des sessions utilisateurs

Fiabilité :

- Disponibilité de la majorité des services ;
- Sauvegarde régulière des données ;
- Gestion appropriée des erreurs ;

Utilisabilité :

- Interface intuitive et ergonomique ;
- Navigation claire et cohérente ;
- Messages d'erreur explicites

Maintenabilité :

- Code documenté et structuré ;
- Architecture modulaire ;
- Respect des bonnes pratiques de développement

2.Choix des technologies

1.1 Technologies Backend

Notre apprentissage étant essentiellement basé sur l'utilisation de Java EE il a été jugé plus qu'utile de l'utiliser pour le développement de notre plateforme principale et ceux pour plusieurs raisons. Le caractère robuste et mature de l'écosystème de Java rend cette implémentation beaucoup plus aisée ; Java « tant l'un des supports natifs des applications web d'entreprise ; la communauté large et la documentation abondante sur Java permet d'avoir facilement l'avis d'autres développeurs pour améliorer son panel technologique et enfin l'évolutivité et la performance qu'offre ce dernier. Plusieurs composants ont été utilisés pour mener à bien ce projet :

Servlets et JSP :

- Les servlets pour la logique métier et le contrôle ;
- JSP pour la génération dynamique des vues ;
- Gestion efficace des requêtes http

Hibernate/JPA

- Mapping objet-relationnel simplifié ;
- Réduction du code SQL manuel ;
- Gestion automatique des transactions ;
- Cache de second niveau pour optimiser les performances

Spring

1.1 Technologies Frontend

La mise en forme des pages web voulant être rendu beaucoup plus dynamique et attractive on a utilisé les principales technologies de développement web

- HTML5 : Structure sémantique des pages web ;

- CSS3 : Styles modernes avec les animations et transitions
- JavaScript : Pour l'interactivité côté client avec la validation des formulaires en temps réel

1.1 Base de données

La gestion de grandes quantités de données nécessite l'utilisation de logiciels adaptés. C'est pourquoi on utilise le Système de Gestion de Base de Données (SGBD) MySQL intégrés avec WampServer pour supporter les transactions, les méthodes CRUD (Create, Read, Update, Delete) et pour la sauvegarde et la récupération des données.



1.1 Serveur d'application



Apache Tomcat

Apache Tomcat a été principalement utilisé durant les phases de développement et de tests. Il s'agit d'un conteneur de servlets léger, rapide à démarrer et simple à configurer. Tomcat prend en charge les technologies essentielles de Java EE telles que **Servlets**, **JSP** et **JSTL**, ce qui le rend parfaitement adapté aux applications web orientées MVC. Sa faible consommation de ressources et sa grande stabilité permettent un déploiement rapide de l'application ainsi qu'une itération efficace lors des phases de développement.

1.1 Outils de développement



- IDE : Eclipse offre un support avancé pour Java EE et Spring, incluant l'autocomplétion intelligente, la gestion des dépendances Maven, le débogage pas à pas, ainsi que l'intégration directe des serveurs d'application. Ils facilitent

également la navigation dans un projet de grande taille et améliorent significativement la productivité des développeurs.



- La gestion du code source a été assurée à l'aide de **Git**, avec un hébergement sur **GitHub**. Cette approche a permis un suivi précis des évolutions du code, une collaboration efficace entre les membres de l'équipe et une gestion rigoureuse des versions. L'utilisation de branches dédiées (feature, develop, main) a facilité l'intégration progressive des fonctionnalités tout en limitant les conflits.

The screenshot shows a MySQL database management interface. At the top, the server is identified as 'MySQL:3306' and the database as 'voyageconnect'. A toolbar contains various icons for database management. Below this is a 'Filters' section with a search box. The main area displays a table list with columns: Table, Action, Rows, Type, Collation, Size, and Overhead. The table list includes: bookings, destinations, invoices, offers, payments, reviews, and users. A summary row shows '7 tables' with a total size of '11.0 KiB'. At the bottom, there is a 'Create new table' dialog with fields for 'Table name' and 'Number of columns' (set to 4), and a 'Create' button. A Windows watermark is visible in the bottom right corner.

Table	Action	Rows	Type	Collation	Size	Overhead
bookings	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_0900_ai_ci	1.0 KiB	-
destinations	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_0900_ai_ci	1.0 KiB	-
invoices	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_0900_ai_ci	2.0 KiB	-
offers	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_0900_ai_ci	1.0 KiB	-
payments	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_0900_ai_ci	1.0 KiB	-
reviews	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_0900_ai_ci	1.0 KiB	-
users	Browse Structure Search Insert Empty Drop	0	MyISAM	utf8mb4_0900_ai_ci	4.0 KiB	-
7 tables	Sum		0 MyISAM	utf8mb4_0900_ai_ci	11.0 KiB	0 B

Figure 1: Table de la Base de données

2. Architecture du système

1.1 Architecture MVC(Model-View-Controller)

Le projet devant être bien structurée et respecter une logique donnée il adopte le pattern architectural MVC qui a pour objectif de séparer les responsabilités en trois composants bien distincts :

- **Model (Modèle) :** il regroupe les entités JPA représentant les données métier (User, Booking...) ; les services métier contenant la logique applicative et le DAO (Data Access Object) pour l'interaction avec la base de données.
- **View (Vue) :** il regroupe les pages JSP pour l'affichage dynamique, les Template HTML, les fichiers CSS pour la présentation sans mentionner les Scripts JavaScript pour l'interactivité.
- **Controller (Contrôleur) :** regroupe les servlets gérant les requêtes http, les controllers Spring MVC et les filtres pour le pré-traitement des requêtes.

1.1 Flux de traitement d'une requête

Le flux de traitement d'une requête dans l'application **VoyageConnect** suit un enchaînement précis d'étapes, garantissant à la fois la sécurité, la cohérence des données et une séparation claire des responsabilités entre les différentes couches de l'application. Tout d'abord, le processus débute lorsque **l'utilisateur effectue une action** depuis l'interface web, telle qu'un clic sur un bouton, la soumission d'un formulaire ou une navigation vers une page spécifique. Cette action déclenche l'envoi d'une **requête HTTP** (GET ou POST) depuis le navigateur vers le serveur. La requête est ensuite **reçue par le serveur d'application Apache Tomcat**, qui joue le rôle de conteneur web. Tomcat se charge d'acheminer la requête vers l'application déployée et de fournir l'environnement d'exécution nécessaire aux composants Java EE.

Avant d'atteindre la logique applicative, la requête passe par le **filtre Spring Security**. Ce dernier vérifie si l'utilisateur est authentifié et s'il dispose des autorisations requises pour accéder à la ressource demandée. En cas d'accès non autorisé, l'utilisateur est redirigé vers la page de connexion ou reçoit un message d'erreur approprié. Une fois les contrôles de sécurité validés, la requête est prise en charge par le **DispatcherServlet**, composant central de Spring MVC. Son rôle est d'analyser l'URL et la méthode HTTP afin de **router la requête vers le contrôleur** correspondant, en se basant sur les annotations de mapping définies dans l'application. Le **contrôleur** agit alors comme intermédiaire entre la vue et la logique métier. Il récupère les paramètres de la requête, effectue les validations nécessaires et **invoque les services métier** appropriés pour exécuter l'action demandée (recherche de voyages, création d'une réservation, consultation de données, etc.). Les **services métier** contiennent les règles fonctionnelles de l'application. Ils coordonnent les différentes opérations et interagissent avec la **couche de persistance** afin d'accéder ou de modifier les données. Cette interaction est réalisée via des repositories ou DAO. À ce niveau, **Hibernate/JPA** se charge de traduire les opérations métier en **requêtes SQL** exécutées sur la base de données. Le Framework gère automatiquement le mapping objet-relationnel, les transactions ainsi que la cohérence des données. Une fois les données récupérées ou mises à jour, les **résultats remontent progressivement** de la couche de persistance vers la couche service, puis vers le contrôleur. Ce dernier prépare alors le **modèle de données** à transmettre à la vue, en sélectionnant les informations nécessaires à l'affichage. La **JSP (Java Server Pages)** est ensuite utilisée pour générer dynamiquement la **réponse HTML** à partir du modèle fourni. Elle intègre les données métier dans l'interface utilisateur, en respectant la structure et le design définis. Enfin, la **réponse HTTP** générée est renvoyée par le serveur au **navigateur client**, où elle est interprétée et affichée à l'utilisateur. Le cycle de traitement de la requête est alors terminé.

2. Modélisation de la base de données

1.1 Diagramme de Cas d'Utilisation

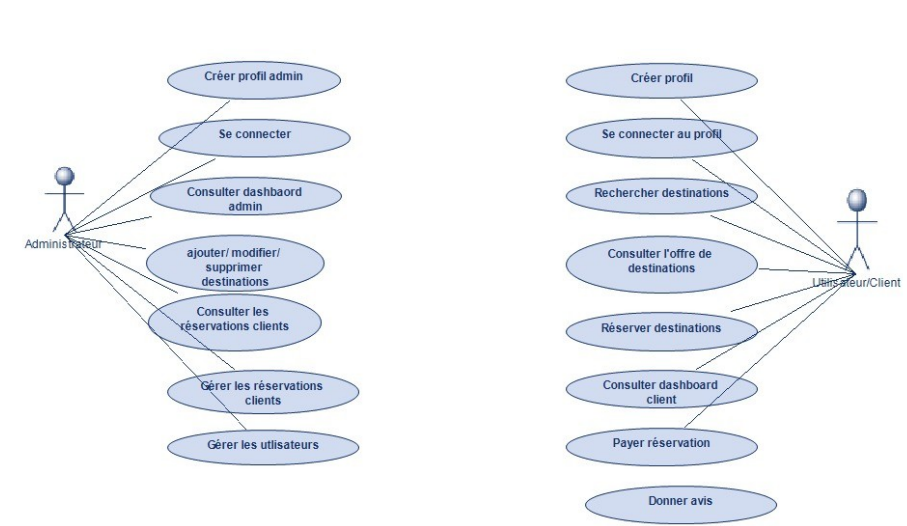


Figure 2: Diagramme de Cas d'Utilisation

1.2 Diagramme de Classe

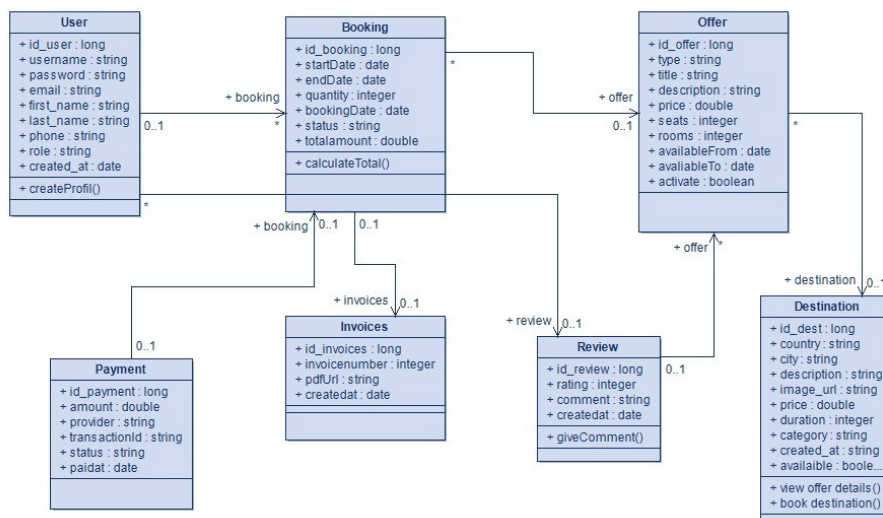


Figure 3: Diagramme de Classe

7. Tests et validation

7.1 Tests unitaires

Les tests unitaires ont pour objectif de vérifier le bon fonctionnement de chaque composant de l'application de manière isolée. Ils portent principalement sur les méthodes métier, les services et les classes utilitaires, sans dépendre de l'interface utilisateur ou de la base de données réelle.

7.1 Tests d'intégration

Les tests d'intégration visent à vérifier le bon fonctionnement de l'application dans son ensemble, en s'assurant que les différentes couches interagissent correctement entre elles. Contrairement aux tests unitaires, ces tests prennent en compte l'intégration réelle entre les contrôleurs, les services, la couche de persistance et la base de données.

Dans VoyageConnect, les tests d'intégration ont été réalisés en exécutant l'application sur un serveur **Apache Tomcat**, avec une base de données configurée pour l'environnement de test. Ils permettent de simuler des scénarios réels d'utilisation, tels que l'inscription d'un utilisateur, la recherche d'un voyage ou la confirmation d'une réservation.

Les différents tests effectués nous ont donné les interfaces que suivent :



Figure 4:Page d'accueil de l'application

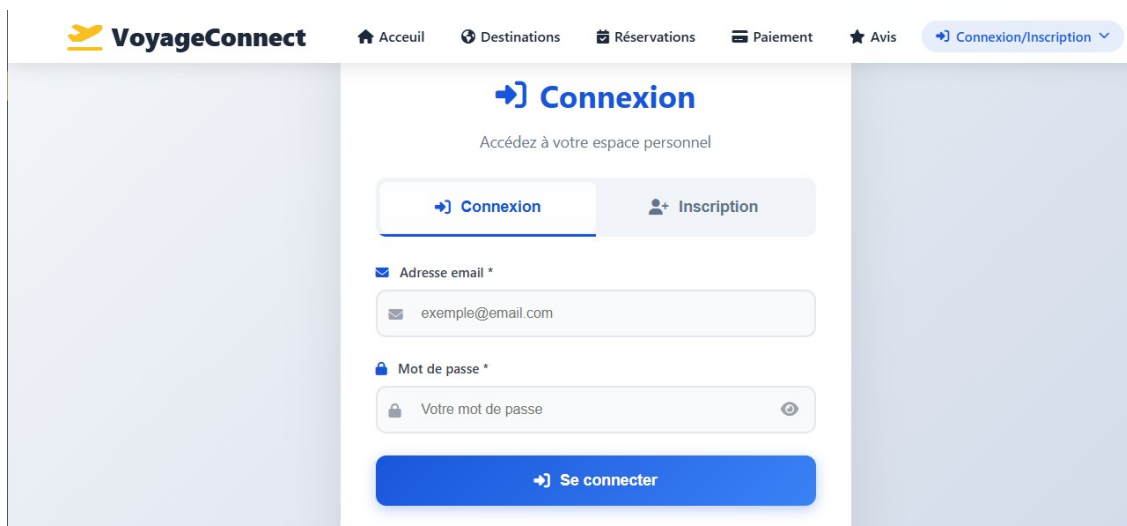


Figure 5:Page de connexion/Inscription Utilisateur

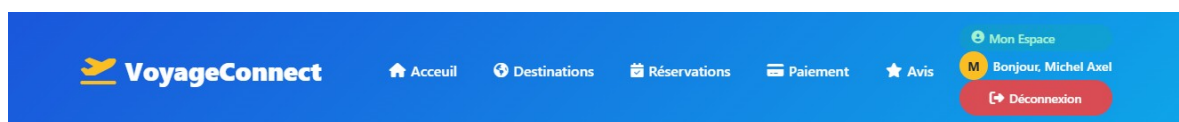


Figure 6: En-tête de l'application après inscription utilisateur

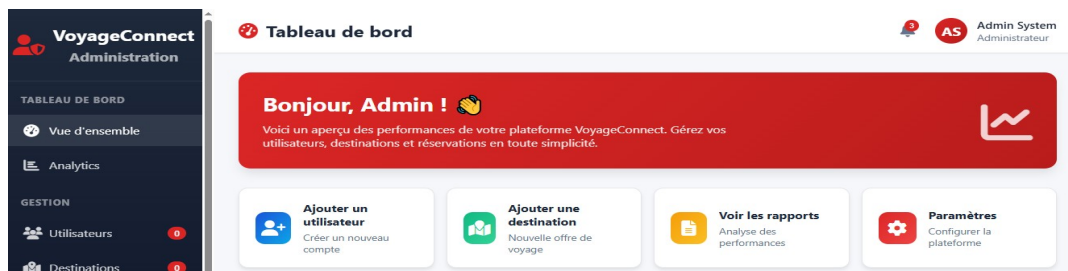


Figure 7:: Tableau de Bord Administrateur

→ Se connecter

+ Créer un compte

→ Connexion Admin

+ Créer un Admin

→ Connexion Administrateur

Accès sécurisé au dashboard d'administration

Email Administrateur

admin@voyageconnect.com

Seuls les emails @voyageconnect.com sont autorisés

Mot de passe

Votre mot de passe administrateur

Figure 8: Connexion Administrateur

8. Difficultés rencontrées et solutions

- ✓ La contrainte du temps insuffisant ne nous a pas permis d'aller au-delà de l'implémentation des fonctionnalités demander pour pouvoir explorer de nouveaux apports de solutions ;
- ✓ L'intégration de la passerelle de paiement n'a pas pu se faire en raison de la complexité de l'intégration et la gestion des erreurs de paiement ;

Nos suggestions

- ✓ L'intégration d'un système de notifications pour informer les utilisateurs sur les nouvelles possibilités de réduction ;
- ✓ Les chats pour permettre aux utilisateurs d'échanger avec des professionnels ;
- ✓ Implémenter la fonctionnalité des avis pour permettre aux utilisateurs de donner leur avis sur leur expérience avec l'appli pour améliorer la qualité

9. Conclusion

À l'issue de ce projet, les objectifs fixés ont été globalement atteints. L'application développée permet aux utilisateurs de rechercher, comparer et réserver différents types de voyages (vols, hôtels, circuits touristiques) à partir d'une interface responsive et ergonomique. Du côté administratif, la plateforme offre des outils performants pour la gestion des offres, des réservations, des utilisateurs et des promotions, facilitant ainsi le pilotage de l'activité de l'agence. Les différentes phases de tests ont permis de valider le bon fonctionnement de l'application, aussi bien sur le plan fonctionnel que sur les aspects de performance, de sécurité et d'utilisabilité. Les retours obtenus lors des tests utilisateurs ont contribué à améliorer l'ergonomie et la clarté du parcours de réservation.

Bien que le projet VoyageConnect réponde aux besoins essentiels d'une agence de voyage moderne, plusieurs pistes d'amélioration et d'évolution peuvent être envisagées. À court terme, l'intégration de nouvelles fonctionnalités telles qu'un système de chat en ligne avec les conseillers de l'agence, une gestion avancée de la fidélité client ou encore une personnalisation plus poussée des recommandations pourrait renforcer l'engagement des utilisateurs. Le développement d'une **application mobile dédiée** (Android et iOS) représenterait également une valeur ajoutée importante afin de toucher un public encore plus large.

Annexes

Annexe A : Guide d'utilisation de l'application web

A.1 Présentation générale

VoyageConnect est une application web permettant aux utilisateurs de rechercher, comparer et réserver des voyages en ligne (vols, hôtels et circuits touristiques). Elle est accessible depuis un navigateur web et propose une interface intuitive adaptée aux ordinateurs, tablettes et smartphones.

A.2 Accès à l'application

1. Ouvrir un navigateur web (Chrome, Firefox, Edge...)
2. Saisir l'URL de l'application
3. Accéder à la page d'accueil

A.3 Connexion/Inscription d'un utilisateur

Sur la page d'Accueil du site l'utilisateur a la possibilité de se connecter s'il a déjà un compte ou de s'inscrire pour créer son profil. En tant que simple utilisateur ou client, il sélectionne « **Profil utilisateur** » dans la liste déroulante du bouton « **Connexion/Inscription** ». Lorsqu'il s'agit d'un administrateur il devra sélectionner « **Profil administrateur** » et se connecter s'il a déjà un compte, sinon s'inscrire d'abord avec les privilèges administrateur.

A.4 Rechercher et consulter les destinations

Sur la page Destination l'utilisateur fait des recherches de destinations dans la barre de recherche ou peut filtrer sa recherche par catégorie (ville, plage, montagne, nature, culture). Ensuite il consulte l'offre détaillée de chaque destination.

A.5 Réserver une ou plusieurs destinations

Après la sélection de destination de son choix et la consultation des détails de l'offre, l'utilisateur peut maintenant faire la réservation soit en cliquant sur le bouton « **Réserver** » de la destination ou soit directement depuis le bouton « **Réserver maintenant** » de la page contenant les détails de l'offre. Pour ce

faire il redirigé directement vers le formulaire de réservation où il renseigne les informations nécessaires puis enfin confirme sa réservation.

A.6 Consulter ses réservations

Après avoir effectué une réservation l'utilisateur peut consulter ses réservations sur la page Réservation ou soit depuis son dashboard auquel il peut accéder depuis le bouton « **Mon Espace** » situé au niveau de son profil de connexion.